Prompt Evolution GA

Brief — what it is and what it found

Explainer generated for Salman Adnan's portfolio

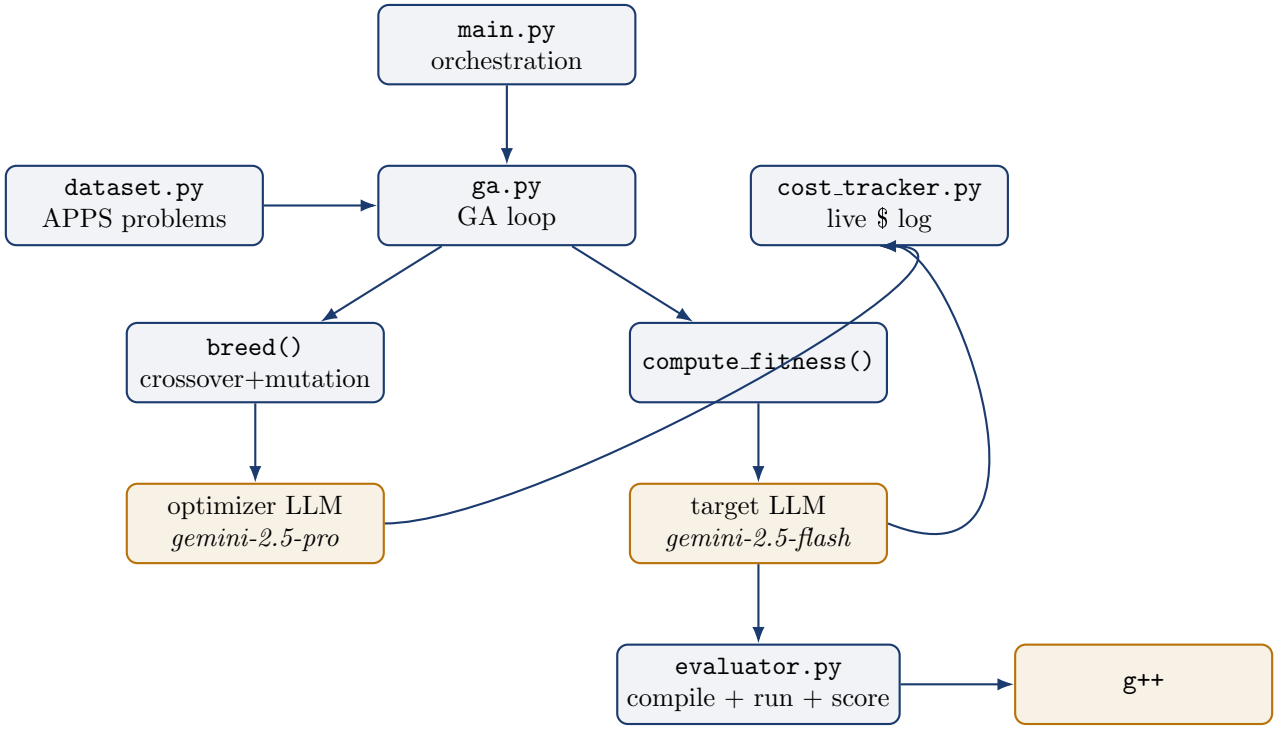
A concise, accurate overview of `prompt-evolution-ga`, for someone who already knows roughly what a genetic algorithm and an LLM are. For a line-by-line walkthrough with every term defined, see the companion `deep-dive.pdf`.

1 What it does

This project runs a **genetic algorithm (GA)** whose chromosome is an entire natural-language **system prompt**, not a bit-string. The goal: automatically discover prompt wording that makes `gemini-2.5-flash` write more correct C++ solutions to competitive-programming problems, replacing manual prompt-engineering trial and error with a search process.

Fitness is objective, not model-judged. Instead of asking an LLM whether generated code "looks correct," each candidate prompt is scored by actually compiling the generated C++ with `g++ -O2 -std=c++23` and running it against a problem's real test cases; fitness is the mean fraction of test cases passed across a 65-problem evolution pool (drawn from the APPS benchmark, Hendrycks et al. 2021). Crossover and mutation are not classic bit operations — they are calls to a second, stronger model (`gemini-2.5-pro`) that edits the prompt text directly.

2 Architecture at a glance



`config.py`	every hyperparameter and model name, in one place
`seeds.py`	15 hand-written starting prompts (population size = 5, so only the first 5 are used unless changed)
`dataset.py`	loads a hand-picked, stratified 300-problem APPS subset; splits it into a 65-problem evolution pool and a 200-problem holdout test set, by a fixed random seed (42)
`llm.py`	all Vertex AI (Gemini) calls, with 10-attempt exponential-backoff retries and strict output-format directives
`evaluator.py`	compiles generated C++, runs it against real test cases, scores it
`ga.py`	the GA loop itself: selection, crossover, mutation, fitness, convergence check, checkpoint/resume
`cost_tracker.py`	per-call token/dollar accounting with a live JSON report
`main.py`	pre-flight checks, orchestration, final holdout evaluation
`run_baseline_eval.py`	evolved-vs-baseline comparison and the 5 result figures
`visualize_test_results.py`	

3 How one generation works

1. **Select** two parents. Three schemes are compared across three separate full runs: *tournament* (pick 3 at random, keep the fittest; default), *roulette* (fitness-proportional random pick), and *tournament + elitism* (like tournament, but the single best individual is always carried over unchanged).
2. **Crossover** (80% of the time): the optimizer LLM merges both parent prompts into one; otherwise one parent is copied as-is.
3. **Mutate** (optional, three independent chances per offspring): inject a new strategy sentence (25%, optionally steered by that lineage's real recent test failures), delete the least useful sentence (15%), or rephrase without changing meaning (10%). The first 2 offspring of every generation always skip mutation, so "pure crossover" children exist to compare against.
4. **Score** every new offspring by running it against all 65 evolution-pool problems (parallelized, 5 worker threads), and repeat for up to 12 generations, with an early-stop rule if the population's fitness spread stays near-zero for 3 generations in a row.

Every generation is checkpointed (population plus RNG state) so a multi-hour run can be safely interrupted and resumed deterministically (`--resume`).

4 Results — the honest headline

Prompt	**Best evolution fitness**	**Holdout mean (200)**
Tournament evolved	0.6531	0.7857
Roulette evolved	0.6619	0.7478
Elitism evolved	0.7346	0.7768
Baseline (best hand-written seed)	—	**0.7900**

No evolved prompt beat the best hand-written seed

On the 200-problem holdout set, every evolved prompt scored below the strongest hand-written seed. Tournament selection came closest and was the only scheme with a positive per-problem record against the baseline (26 wins, 22 losses, 152 ties). This is the project's own stated conclusion, not a softened or inflated version of it.

Cost across all three full 12-generation runs: roughly 13,000 LLM calls, about $17.05 total (Vertex AI, Gemini pricing). Five `matplotlib` figures in `results/comparison/` back this comparison: overall score, outcome-reason breakdown, score by difficulty rating, per-problem scatter vs. baseline, and win/tie/loss counts — all regenerated deterministically from raw evaluation logs by `visualize_test_results.py`.

5 Why it didn't work better — key limitations

- **Population of 5 is very small** for 12 generations; a single offspring can swing the population mean, and the scheme-to-scheme test gaps sit inside that noise.
- **The 700-token operator cap truncates prompts.** The tournament run's winning prompt literally ends mid-sentence ("... variable types, edge"), and still won its run — operator output was never validated for completeness.
- **No statistical significance testing** on the evolved-vs-baseline gap (a fraction of a percentage point over 200 problems).
- **Exact-match test scoring** penalizes correct-but-differently-formatted answers (e.g. floating-point precision), adding noise to every fitness score.
- **Semantic drift:** because crossover/mutation are LLM edits of natural language rather than fixed-structure operations, offspring can wander away from the strategies that made their parents fit — the reason genealogy (parents, operators, intermediate prompt states) is logged for every individual.

Discrepancy flagged, not resolved

The accompanying course paper's abstract states evolved prompts "outperform hand-crafted baselines... achieving meaningful improvements in pass@1 scores," but the results actually committed in this repository show the opposite on the 200-problem holdout (table above). This mismatch is flagged for the owner rather than silently reconciled — it may reflect an earlier/smaller comparison the abstract was written against, or simply an optimistic framing predating the full test run; the source data does not say which.

6 Engineering worth noting

- Parallel fitness evaluation via `ThreadPoolExecutor` (5 workers).
- Full genealogy logging: every individual records its parents, which operators produced it, and the prompt text after each intermediate step.
- Per-generation checkpoints (including Python's RNG state) enabling exact, deterministic `--resume`.
- Live, per-call cost tracking (tokens + USD) written to disk during a run so an in-progress run can be watched or killed if costs drift.

- A hard pre-flight check (credentials, dataset, dependencies) that fails fast with a clear message rather than partway through a multi-hour run.

7 Glossary (abbreviated)

Term	Meaning
Chromosome	The candidate solution evolved; here, an English prompt string.
Fitness	A score for a candidate; here, mean test-case pass rate.
Crossover / mutation	Combining two parents / applying a small change to an offspring.
Pass@1	Standard metric: does the model's one attempt pass.
Holdout set	Data never used during evolution, reserved for the final honest test.
Vertex AI	Google Cloud's managed platform for calling hosted LLMs.
Thread pool	A fixed set of worker threads running tasks concurrently.